

Cloud Computing (cloud)

Module 10: Virtualization Revisited in the Context of Kubernetes

Prof. Dr. Sebastian Graf (sebastian.graf@fhnw.ch)

Norwin Schnyder (norwin.schnyder@fhnw.ch)



Agenda

Virtual Machine Management on Kubernetes (KubeVirt)

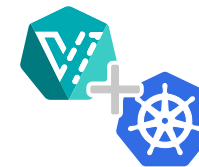
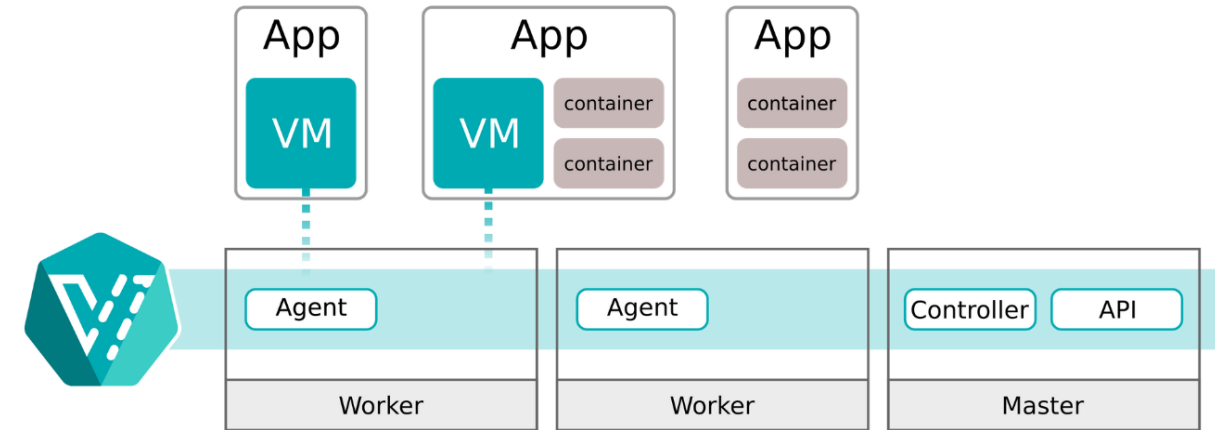
Labs 01 - KubeVirt quickstart with kind

Labs 02 - Experiment with CDI

KubeVirt

KubeVirt is a *Kubernetes-native* approach to virtualization, allowing virtual machines to run alongside containers in the same cluster:

- Declarative VM Management (Kubernetes API is Extended using CustomResourceDefinitions)
- Integration with Kubernetes Ecosystem
 - Storage
 - Networking
 - Observability (Metrics & Logs)

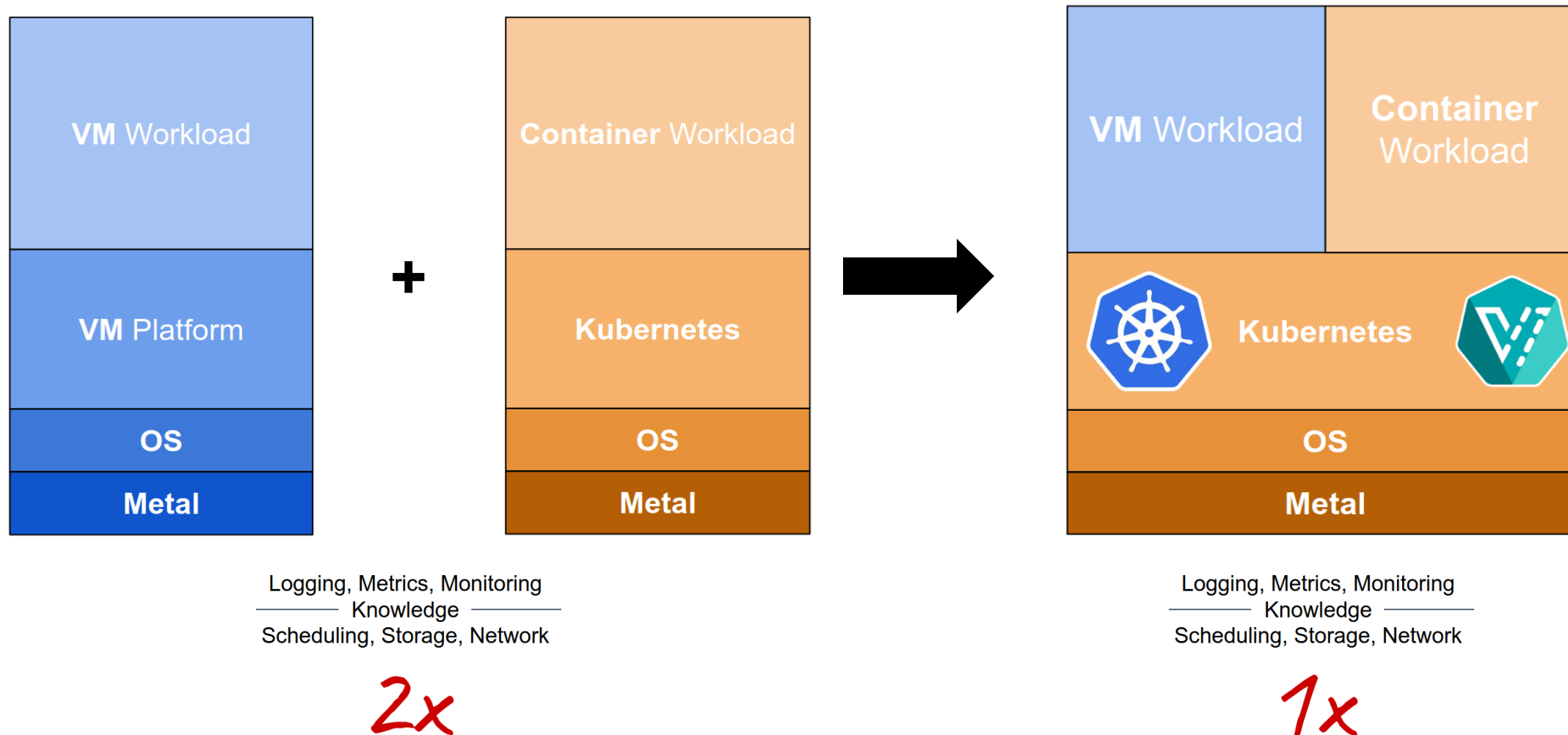


Leverage KubeVirt and Kubernetes to manage virtual machines for impractical-to-containerize apps (e.g., appliances with customer kernels).

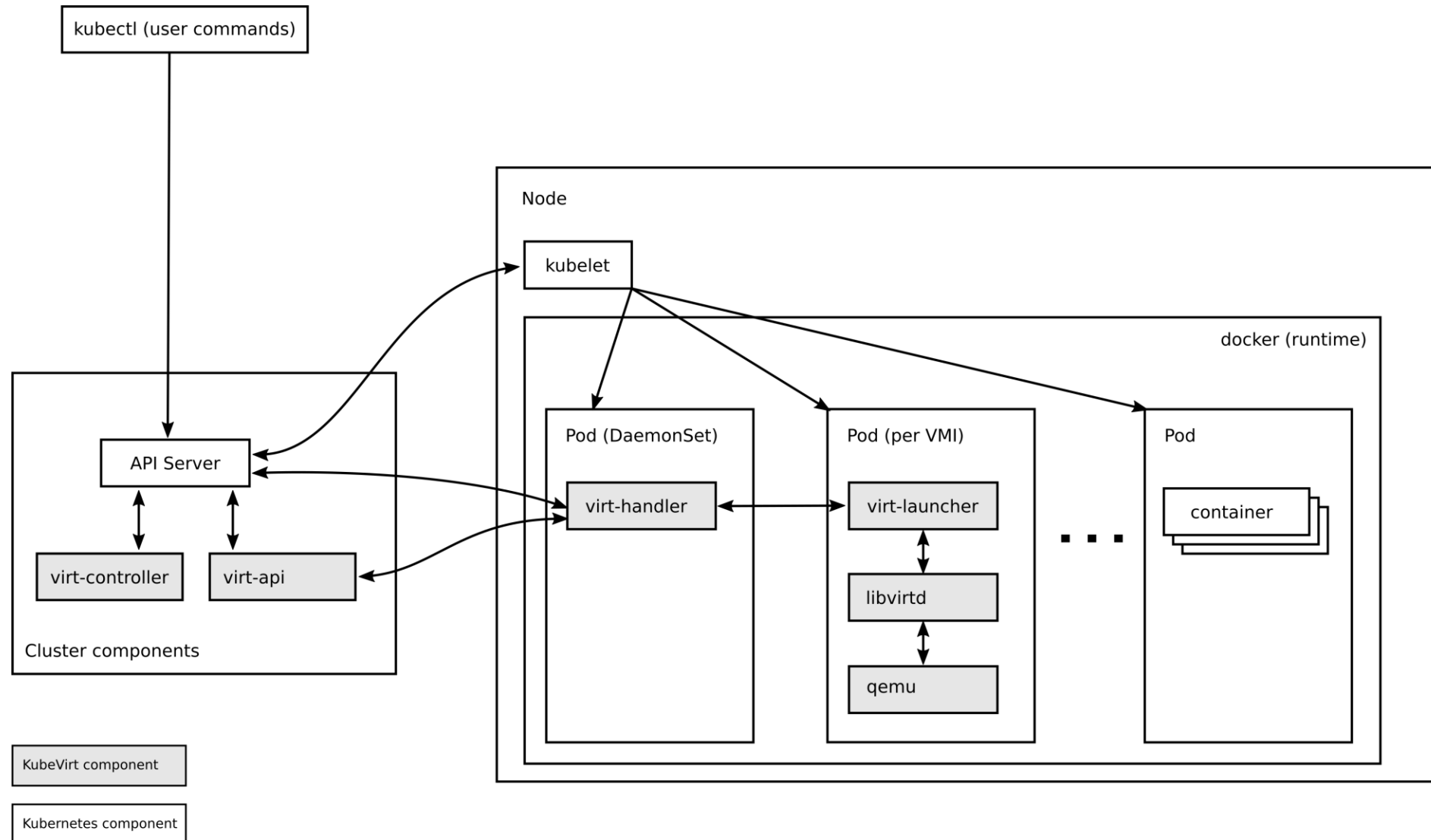


Combine existing virtualized workloads with new container workloads on the one platform.

KubeVirt: Infrastructure & Workflow Convergence



KubeVirt System Architecture



KubeVirt Components: virt-api-server

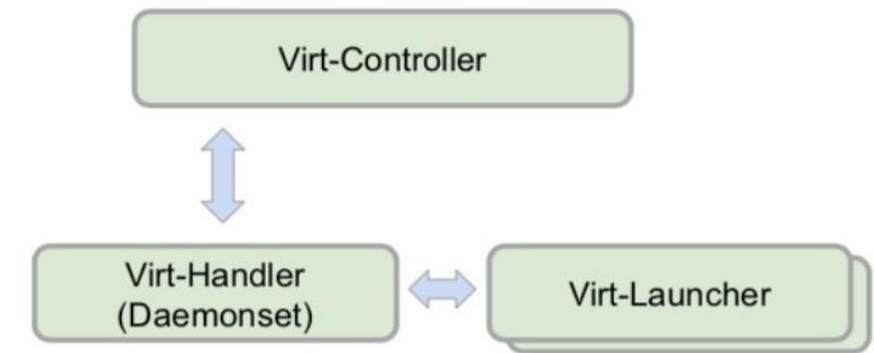
- HTTP API server which serves as the **entry point** for all virtualization related flows.
- The API Server is taking care to update the virtualization related custom resource definition (CRD).
- As the main entrypoint to KubeVirt it is responsible for **defaulting** and **validation** of the provided VMI CRDs.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstance
metadata:
  name: testvmi-nocloud
spec:
  terminationGracePeriodSeconds: 30
  evictionStrategy: LiveMigrate
  domain:
    resources:
      requests:
        memory: 1024M
    devices:
      disks:
        - name: containerdisk
          disk:
            bus: virtio
        - disk:
            bus: virtio
            name: cloudinitdisk
  volumes:
    - name: containerdisk
      containerDisk:
        image: kubevirt/fedora-cloud-container-disk-demo:latest
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |-
          #cloud-config
          password: fedora
          chpasswd: { expire: False }
```

VMI definitions are kept as custom resources (CR) inside the Kubernetes API server. It is defining all properties of the Virtual machine itself.

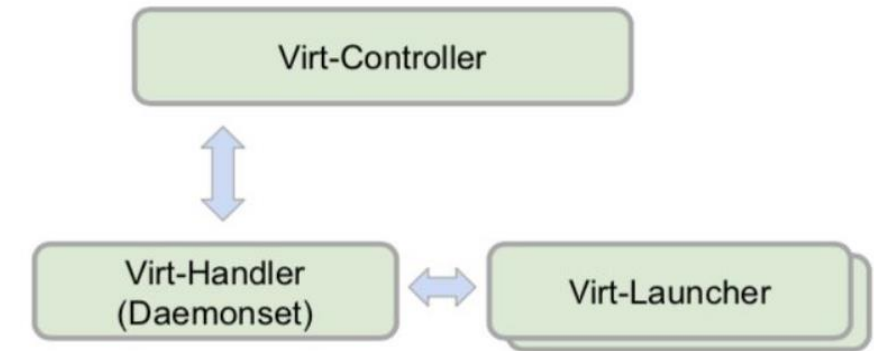
KubeVirt Components: virt-controller

- From a high-level perspective the virt-controller has all the *cluster wide* virtualization functionality.
- This controller is responsible for monitoring the VMI (CRs) and managing the associated pods. Currently the controller will make sure to create and manage the life-cycle of the pods associated to the VMI objects.
- A VMI object will always be associated to a pod during it's life-time, however, due to i.e. migration of a VMI the pod instance might change over time.



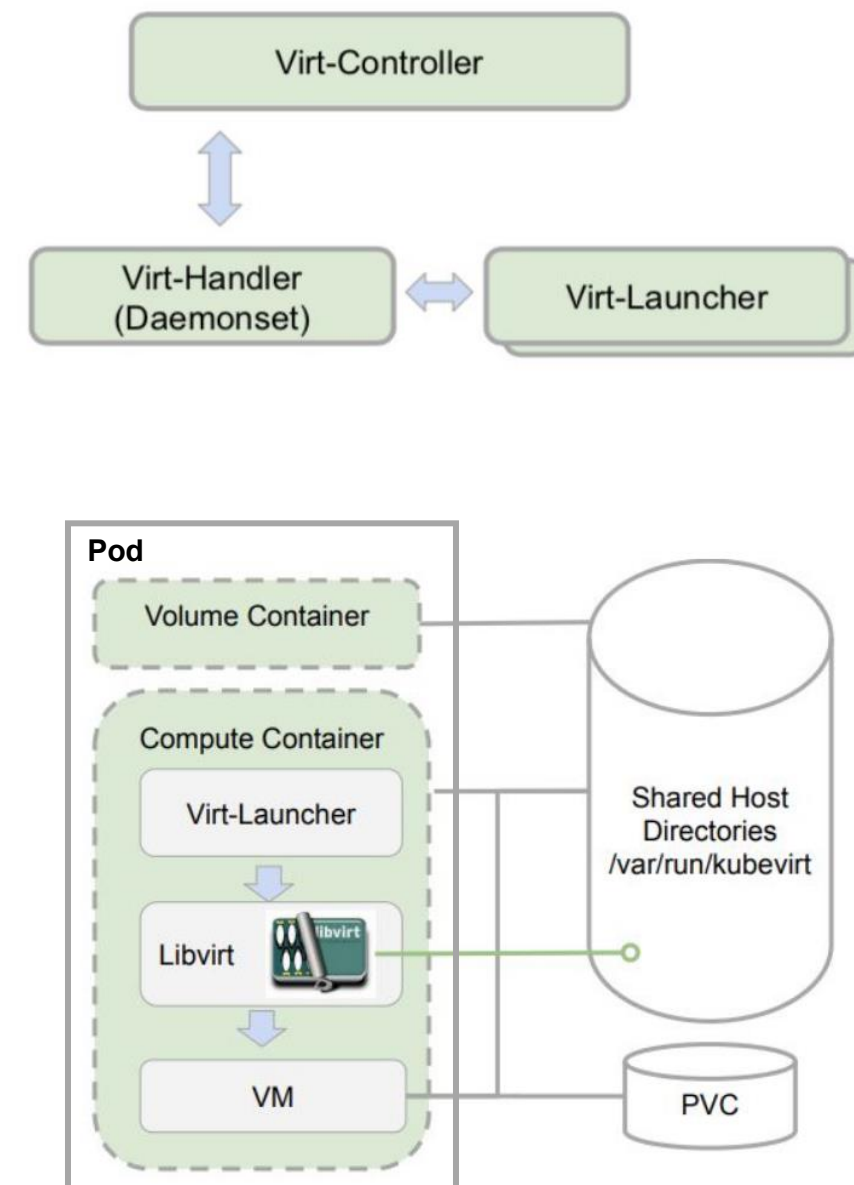
KubeVirt Components: virt-handler

- Every host needs a single instance of virt-handler, delivered as a DaemonSet.
- Like the virt-controller, the virt-handler is also reactive and is watching for changes of the VMI object, once detected it will perform all necessary operations to change a VMI to meet the required state.
- This behavior is similar to the choreography between the Kubernetes API Server and the kubelet.
- The main areas which virt-handler has to cover are:
 1. Keep a cluster-level VMI spec in sync with a corresponding libvirt domain.
 2. Report domain state and spec changes to the cluster.
 3. Invoke node-centric plugins which can fulfill networking and storage requirements defined in VMI specs.



KubeVirt Components: virt-launcher

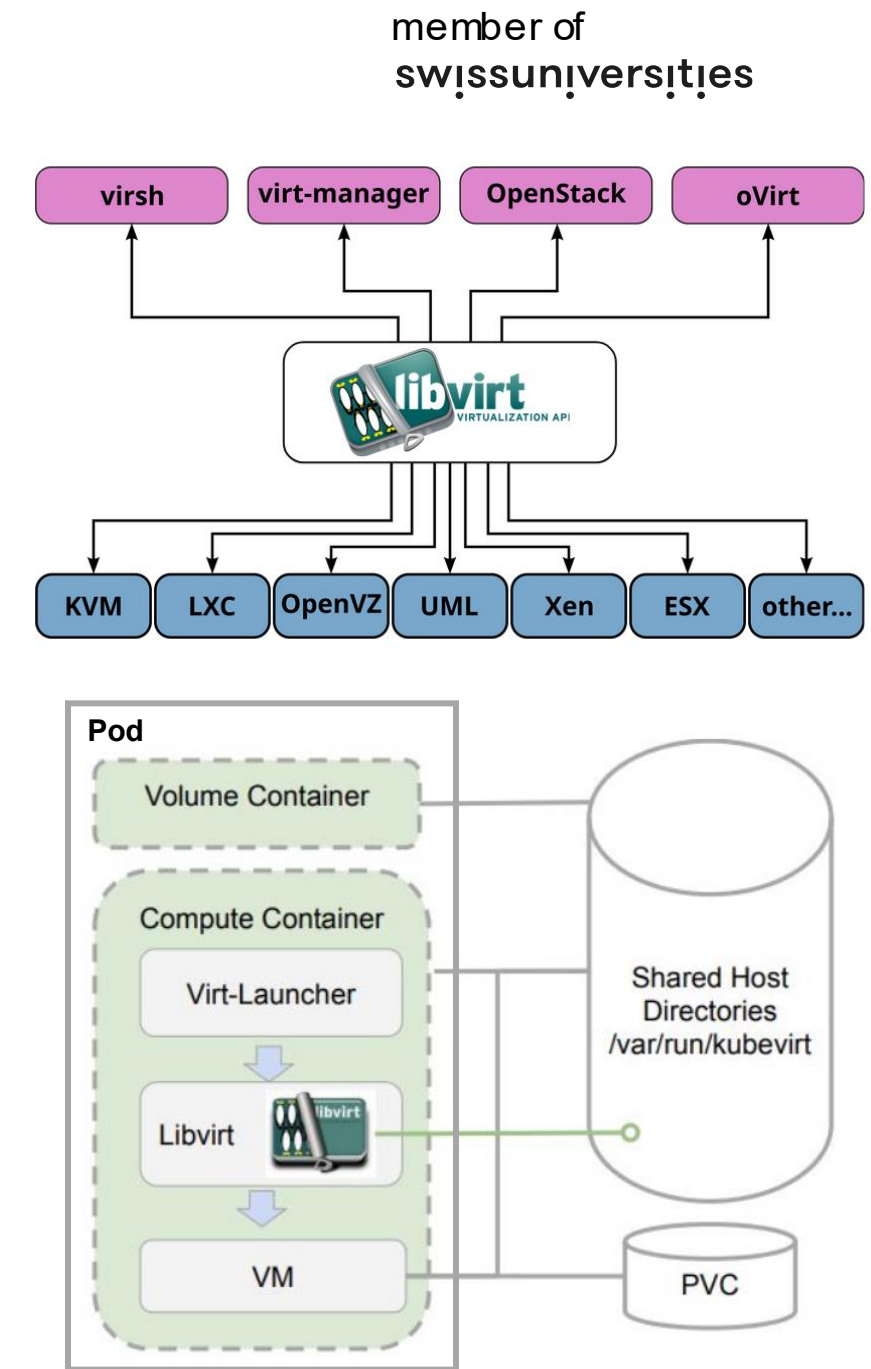
- For every VMI object one pod is created. This pod's primary container runs the virt-launcher KubeVirt component.
- Kubernetes or the kubelet is not running the VMs itself. Instead a daemon on every host in the cluster will take care to launch a VMI process for every pod which is associated to a VMI object whenever it is getting scheduled on a host.
- virt-handler signals virt-launcher to start a VMI by passing the VMI's CRD object to virt-launcher. virt-launcher then uses a local **libvirt** instance within its container to start the VMI.
- VM metrics are collected through **libvirt** APIs and reported using a prometheus endpoint.



libvirt: The virtualization API

The **libvirt** project is an open source toolkit to manage virtualization platforms. It is written in C, but is also accessible from Python, Perl, Go.

- An instance of libvirtd is present in every VMI pod.
- virt-launcher uses libvirtd to manage the life-cycle of the VMI process.



KubeVirt System Architecture

